Presubmit Rescue: Automatically Ignoring Flaky Test Executions

Minh Hoang
Google Inc.
Mountain View, CA, USA
hoanhminh@google.com

Adrian Berding
Google Inc.
Mountain View, CA, USA
aberding@google.com

ABSTRACT

Flaky tests often give misleading signals and slow down the speed at which Software Engineers can develop. Traditional ways to help mitigate the developer time loss revolve around identifying tests which are flaky and choosing not to run them or adding additional attempts to reduce the chance of them flakily failing. We present Google's newest approach to the problem that tries to maintain the usefulness of flaky tests without being as time and resource wasteful as executing additional test attempts (deflaking). Presubmit Rescue is a machine learning model that predicts if a failed test execution was caused by a flake. Predicted flaky failures can be automatically ignored to save developer time without spending machine time for retries or developer time on debugging.

ACM Reference Format:
Minh Hoang and Adrian Berding. 2024. Presubmit Rescue: Automatically Ignoring Flaky Test Executions. In *2024 International Flaky Tests Workshop 2024 (FTW '24), April 14, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3643656.3643896

1 INTRODUCTION

Tests are a fundamental part of the code development cycle at Google not unlike most modern companies and open source repositories. The life cycle of a change at Google goes through several phases of testing that progressively try to be more complete than the previous [5].

(1) Developers manually run local tests close to their change
(2) Developers run tests associated with projects that are configured to run prior to committing the change (TAP Presubmit)
(3) All tests run continuously at depot head (TAP Postsubmit)

Presubmit has a big impact on a developer's day to day workflow since it can directly gate a change from submitting, it is also often used by developers for verifying their change is not breaking any tests before requesting a peer review. The nature of Google's mono repository often leads to Presubmit testing being more extensive than one might find at other companies since the cost of a bad change directly affects the main branch for all affected projects [3, 4].

A TAP Presubmit with all passing tests (green) can give developers and reviewers reassurance that the code change is unlikely to break any projects. On the other hand, a TAP Presubmit with failing tests (red) signals to the developer that their change may require edits. However a red TAP Presubmit run does not always indicate a problem in the pending change, things like already broken projects or flaky tests often give developers a false signal which wastes their time debugging or even gates a "good" change from submission.

When testing at Google scale, it is common to get a false red signal (friction) during Presubmit. In particular, flaky tests cause half of the friction engineers experience at Presubmit time. Existing solutions for flaky tests revolve around identifying what tests have flaked in the past and doing one or more of the following actions:

(1) Providing feedback on the tests and asking developers to fix them.
(2) Not running them in Presubmit (skipping).
(3) Always running them with multiple attempts.

Asking developers to fix flaky tests seems like the ideal solution, however in practice developers often do not fix tests either because they have other priorities or fixing them is non-trivial. Moreover, engineers often get friction from running tests that they do not own and should not be responsible for fixing. Skipping flaky tests might be the most optimal solution if we believed that flaky tests provide no value, however, many depot breakages are caused by skipping flaky tests [2] especially for projects where tests are often inherently flaky like distributed systems that have large scale integration tests. Causing breakages can lead to more developer slowdown and counteract the efforts of skipping tests. Lastly, adding more attempts to known flaky targets gives the tests a lower chance of flaking, however this approach can be very costly [1]. It often relies on making the decision to do in-parallel attempts which is costly of machine resources or doing sequential attempts that can be costly of developer time.

At Google prior to the topic of this paper we used a combination of all 3 methods, skipping flaky tests only for projects that have high amounts of friction. Even with all these approaches, Googlers were still dealing with high amounts of flakes in Presubmit, in fact an average Presubmit had a 3.5% chance to fail do to flakes. Ideally we needed something that could be as effective as skipping tests without costing as many breakages as skipping test and not costing as many resources as simply rerunning tests.

2 METHOD OVERVIEW

2.1 Architecture Overview

Presubmit Rescue is a gradient-boosted decision tree model that outputs a confidence level indicating how likely a given test execution failed due to flakiness. Unlike previous Google approaches, this model focuses on the current executions rather than predicting the probability of a test flaking in the future. Because of this distinction, the model runs after tests finish executing, but before we report the results back to the engineers.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
FTW '24, April 14, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0558-8/24/04.
https://doi.org/10.1145/3643656.3643896

If Presubmit Rescue is highly confident that a failure was just a flake, then we will "ignore" the result. When all failing results are ignored, then the TAP Presubmit will turn green and the change submission will go through. In some cases when Presubmit Rescue is not confident enough to just ignore, we also automatically retry them. This retry approach is different from the existing solution of retrying failures with known flakiness history mentioned in the introduction section because one happens before the test execution and the other after. The added information we get after the initial executions allows us to be more efficient and smart about devoting resources to deflaking. For example if the majority of tests fail in the initial execution of a TAP Presubmit, it often does not make sense to waste time to deflake since there is likely something wrong with the underlying change.

To support Presubmit Rescue, we have 3 major components running continuously. One of them is the serving stack mentioned earlier in this section that sits between executing tests and reporting results back to the engineers. The main responsibility of this stack is generating inputs for the model, which involves sending requests to multiple services or querying databases. Being responsive is vital for this component because we do not want to add latency to the development workflow. Another major component of our infrastructure is the training stack, which takes in raw data from the serving side, labels it, trains, and releases multiple models constantly. Lastly, we have a stack that listens to all test and build executions at Google and tries to extract as much useful information from them as possible. For example, it populates materialized views so that we know how often a certain test fails within the last 1 hour as part of a TAP Presubmit.

2.2 Training Data and Features

To label our training data, our system keeps track of all failures that occur for any TAP Presubmit throughout the day and runs them again under the same conditions up to 15 times at night when resource usage is under capacity. If a failure ever eventually passes, then we label the original execution as a flaky example. The remaining are either failures due to the change being tested (real failures), highly flaky tests that we have not retried enough yet, or existing failures that have been occurring before the change.

To make recommendations, we utilize multiple variations of the Presubmit Rescue, such as a model trained on only test failures or one trained on only build failures. The following are a few of the most impact-full features that go into the models:

- The history of the test, for example, how many times it fails or passes for the last 1 hour. We found that the model performs better with a lot of time windows, such as last 7 days, 30 days, or even as granular as within the same change or for the same user.
- The resource usage of the test, such as how much RAM or how long the test took. Flaky failures often take longer to fail than a real failure.
- The error message and how similar it is to that of known flakes. We have a service that clusters flakes with similar error messages together, so that the model can deal with common flakiness root causes such as timed out requests sent to a live server.
- The context for the code change. Is the commit adding only comments? Are there other failures alongside with the one we are giving a recommendation for? We found that real failures often fail together in a cluster, but flakes are more sparse. Another good context feature is the distance between the file being changed and the test itself.

3 EVALUATION

We train our models on 3 months of failures, which add up to over 30 millions failed test executions, and then evaluate these models in parallel with live real data. Only about 3% of these failures will get a flaky label from our 15 nightly deflaking runs, making the problem space heavily skewed towards non-flaky failures.

Giving an incorrect recommendation for a failure can result in a real failure being ignored and if this failure was the only one blocking the change, then the test would consistently fail in our CI stack. These incidents are fairly costly, so we set thresholds on our model to maximize catching as many flakes as possible without the precision falling below a static number we have set. On a failure-level, we are able to achieve an AUPRC of over 0.9. Since multiple failures can gate a change from submitting, it is also important for us to keep track of the rate on a Presubmit-level as well. Our model is able to successfully unblock over 80% of Presubmits failed solely due to flakes at a cost of approximately 5 breakages per week.

Compared to existing solutions, if Google were to skip what we classify as the "most" flaky tests, then we can only unblock about 10% of Presubmits that failed due solely to flakes at the cost of about 300 breakages per week. On a failure-level, skipping known flakes is correct only half of the time. Also, determining which test is flaky is very challenging too because for those that do not flaky fail often, then our CI stack would take multiple days or weeks to see one of the flaky executions.

REFERENCES

[1] Gabin An, Juyeon Yoon, Thomas Bach, Jingun Hong, and Shin Yoo. 2023. Just-in-Time Flaky Test Detection via Abstracted Failure Symptom Matching. arXiv:cs.SE/2310.06298

[2] Guillaume Haben, Sarra Habchi, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2023. The Importance of Discerning Flaky from Fault-triggering Test Failures: A Case Study on the Chromium CI. arXiv:cs.SE/2302.10594

[3] Atif Memon, Zebao Gao, Bao Nguyen, Sanjeev Dhanda, Eric Nickell, Rob Siemborski, and John Micco. 2017. Taming Google-scale Continuous Testing. In *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. IEEE, Piscataway, NJ, USA, 233–242. https://doi.org/10.1109/ICSE-SEIP.2017.16

[4] Rachel Potvin and Josh Levenberg. 2016. Why Google Stores Billions of Lines of Code in a Single Repository. *Commun. ACM* 59, 7 (2016), 78–87. https://doi.org/10.1145/2854146

[5] Peter Spragins. 2018. Efficacy Presubmit. https://testing.googleblog.com/2018/09/efficacy-presubmit.html